The Scaled Dot-Product Attention Algorithm

Nazmul Hasan (ID: 2023100000130)

Department of Computer Science, Southeast University

Algorithm Overview

The Scaled Dot-Product Attention algorithm is the foundation of modern sequence processing. This algorithm computes the contextual relationships between items in a sequence by processing the entire input simultaneously rather than sequentially. It works by taking a target element (the Query) and comparing it against a set of Key-Value pairs that represent the surrounding context. By calculating similarity scores, the algorithm assigns higher weights to the most relevant elements. This allows the system to easily capture long-range dependencies and understand the true meaning of a sequence, even if related words are far apart.

Mathematical Formulation

The Attention mechanism operates entirely through linear algebra and matrix multiplications. The foundational equation that dictates the algorithm's behavior is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Variable Definitions:

- Q **(Query Matrix):** Mathematical representation of the current sequence item.
- K **(Key Matrix):** Representations of all items serving as context.
- V **(Value Matrix):** Matrices holding the actual semantic informational content.
- d_k **(Dimension):** Vector length of the keys, used as a scaling denominator.
- QK^T**:** Dot product calculating raw similarity between Query and Keys.

3. Step-by-Step Execution: Matrix Flow

Visual Simulation: "Bank River"

To see how the algorithm executes, we simulate a 2-word sequence. We want the algorithm to learn that "Bank" is related to water.

Step 1: MatMul (Calculating Raw Scores)

The algorithm multiplies Query (Q) by transposed Key (K^T) to find the raw alignment score. Notice the high score (30) between Bank and River.

$$Q \times K^T = \begin{bmatrix} q_{\text{bank}} \\ q_{\text{river}} \end{bmatrix} \times \begin{bmatrix} k_{\text{bank}} & k_{\text{river}} \end{bmatrix} = \begin{bmatrix} 10 & 30 \\ 5 & 20 \end{bmatrix}$$

Step 2: Scale & SoftMax (Converting to Weights)

Scores are divided by $\sqrt{d_k}$ and passed through Softmax to normalize rows into probabilities. Result: "Bank" pays **90% of its attention** to "River".

$$\text{Softmax}\left(\frac{\text{Scores}}{\sqrt{d_k}}\right) = \begin{bmatrix} 0.10 & \mathbf{0.90} \\ 0.20 & 0.80 \end{bmatrix}$$

Step 3: MatMul with V (Final Context)

Probability weights are multiplied by the Value (V) matrix. The embedding for "Bank" is now heavily infused with the meaning of "River".

$$\begin{bmatrix} 0.10 & 0.90 \\ 0.20 & 0.80 \end{bmatrix} \times \begin{bmatrix} v_{\text{bank}} \\ v_{\text{river}} \end{bmatrix} = \begin{bmatrix} 0.10(v_{\text{bank}}) + \mathbf{0.90}(\mathbf{v}_{\text{river}}) \\ 0.20(v_{\text{bank}}) + 0.80(v_{\text{river}}) \end{bmatrix}$$

4. Diagrams & Visualizations

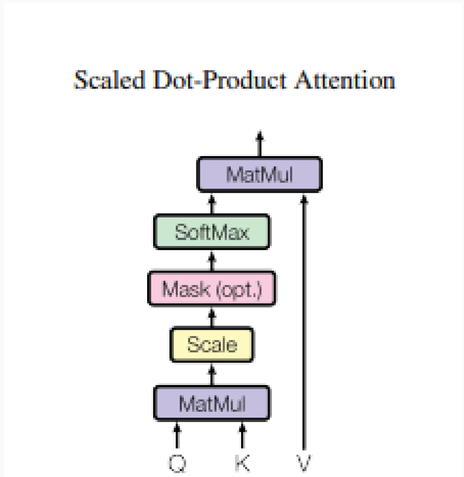

Fig 1: Scaled Dot-Product Attention architecture (Vaswani et al., 2017).

5. Time & Space Complexity

Let N = Sequence Length and d = Embedding Dimension.

1. Time Complexity Breakdown: $\mathcal{O}(N^2 \cdot d)$

- **Phase A: Alignment Scores** ($Q \times K^T$)
  Shapes: $[N \times d] \times [d \times N] \rightarrow [\mathbf{N} \times \mathbf{N}]$. Requires d multiplications across N^2 elements. Cost: $\mathcal{O}(N^2 \cdot d)$
- **Phase B: Scaling and Softmax**
  Scales and normalizes the resulting grid. Cost: $\mathcal{O}(N^2)$
- **Phase C: Contextual Output (Weights** $\times V$**)**
  Shapes: $[N \times N] \times [N \times d] \rightarrow [\mathbf{N} \times \mathbf{d}]$. Cost: $\mathcal{O}(N^2 \cdot d)$

Conclusion: Quadratic scaling (N^2) is why processing extremely long documents requires massive computational power.

2. Space (Memory) Complexity: $\mathcal{O}(N^2 + N \cdot d)$

- **Storing the Inputs:** Holding Q, K, and V matrices costs $\mathcal{O}(N \cdot d)$.
- **The Bottleneck:** The algorithm must instantly instantiate and store the entire $N \times N$ raw alignment grid in memory before it can apply Softmax. Cost: $\mathcal{O}(N^2)$.

6. Real-World Application: Machine Translation

Resolving Polysemy in NMT Systems

When translating *"I sat on the bank of the river"* into Bengali, sequential models often confuse "bank" as financial (ব্যাংক) vs. river side (নদীর তীর). SDPA resolves this contextually:

6. Real-World Application (Continued)

Step-by-Step Execution:

1. Querying: Sets the ambiguous word "bank" as the active Query (Q).
2. Contextual Scanning: Compares Q against all words (Keys). Math finds the highest alignment match between "bank" and "river."
3. Semantic Fusion: Updates the vector for "bank" to include the data for "river."
4. Output: The system confidently translates to: (নদীর তীর).

7. Case Study & Matrix Simulation

Coreference Resolution (The Pronoun Problem)

"The animal didn't cross the street because ***it*** *was too tired."*

How does the computer know **"it"** refers to the animal?

Step A: The Dot Product ($Q \times K^T$)

The Query for "it" (q_{it}) finds strong alignment with "animal" (k_{animal}) because the context contains "tired".

$$q_{\text{it}} \times \begin{bmatrix} k_{\text{animal}} & k_{\text{street}} & k_{\text{it}} \end{bmatrix} = \begin{bmatrix} \mathbf{5.0} & 1.0 & 2.0 \end{bmatrix}$$

Step B: Softmax Normalization

The dominant score ($e^{5.0}$) results in **93% attention** assigned to "animal".

$$\text{Softmax}(\dots) = \begin{bmatrix} \mathbf{0.93} & 0.02 & 0.05 \end{bmatrix}$$

Step C: Semantic Fusion

Weights are multiplied by Value vectors (V). Result: "it" is now **93% animal**.

$$\begin{bmatrix} \mathbf{0.93} & 0.02 & 0.05 \end{bmatrix} \times \begin{bmatrix} v_{\text{animal}} \\ v_{\text{street}} \\ v_{\text{it}} \end{bmatrix} = \mathbf{0.93}(\mathbf{v}_{\text{animal}}) + \dots$$

8. Algorithm Comparison

Feature	CNN	RNN	LSTM	Self-Attention (Transformer)
Primary Use	Image processing	Sequential data	Long-term dependencies	NLP, vision tasks
Processing Style	Spatial hierarchies	Sequential	Sequential with memory	Parallelized
Long-Term Context	No	Limited	Yes	Yes (Simultaneous)
Training Speed	Fast	Slow	Slower	Fast (parallel matrix math)
Scalability	High	Low	Moderate	Very high

References

1 Vaswani, A., et al. (2017). Attention is all you need. *NIPS*, 30.

2 Devlin, J., et al. (2018). BERT. *arXiv:1810.04805*.

3 Smith, E. "CNN vs RNN vs LSTM vs Transformer". *Medium*.